

International Cybersecurity and Digital Forensics Academy (ICDFA)**CIP-B103: Network Forensics Fundamentals****LAB 5: ARP Poisoning Forensics****Three-Week Delivery Block 2/3 | 25-26 July 2026**

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	CIP-B103
Course Title	Network Forensics Fundamentals
Lab Number	Lab 5
Lab Title	ARP Poisoning Forensics
Students Name	Fuseini Imoru Kantuogaa
Student ID	17291
Instructor Name	Dr. Sarah James
Delivery Block	2/3 of 3
Scheduled Dates	25-26 July 2026
Estimated Duration	4-5 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	arp.pcap and an isolated host-only ARP capture

1. Source Materials and Slide Alignment

- Address Resolution Protocol Forensics: ARP operation, ARP caches, request/reply fields, capture with tshark, ARP spoofing/poisoning, man-in-the-middle consequences and evidence analysis.
- The slide practical compares clean and poisoned ARP tables and uses a supplied arp.py script only within the same LAN.

- The assignment requires legitimate ARP capture, spoofing capture and proof of a poisoned ARP table.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded CIP-B103 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

A user reports that the default gateway intermittently resolves to the wrong MAC address. You will first document normal ARP resolution, then analyze a known ARP-poisoning capture. In an instructor-approved host-only network, you may reproduce a short controlled poisoning event, capture the evidence, restore the ARP mappings and explain how the attack is detected.

3. Learning Outcomes

- Explain how ARP maps IPv4 addresses to MAC addresses on a LAN.
- Inspect ARP caches on Linux and Windows systems.
- Identify ARP requests, legitimate replies and unsolicited or conflicting replies.
- Detect one IP address being associated with multiple MAC addresses.
- Capture and document a bounded ARP-poisoning simulation in a host-only network.
- Restore network state and recommend preventive controls.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.
- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- The live component is optional and must use instructor-assigned RFC1918 addresses on a host-only/internal virtual switch. Never poison a home, school, office, hotel or public Wi-Fi network.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Virtual machines	Kali attacker/analyst plus Linux or Windows victim	Create an isolated LAN scenario.
Network mode	Host-only/internal network	Prevent access to production networks.
Capture tools	Wireshark and tshark	Capture ARP packets.
Packet library	python3-scapy	Run authorized training scripts.
System tools	ip neigh, arp, ipconfig /all	Inspect ARP tables and interfaces.
Evidence	arp.pcap or student capture	Analyze known poisoning indicators.

6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/CIP-B103-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab5
pwd
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y wireshark tshark python3-scapy net-tools
wget -O evidence/arp.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-
lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_table_initial.txt
```

Screenshot required: Interface addresses, route/gateway, initial ARP table and evidence hashes.

```
(kali㉿kali)-[~]
$ mkdir -p ~/CIP-B103-Lab5/{evidence,working,exported,reports,screenshots,scripts}

(kali㉿kali)-[~]
$ cd ~/CIP-B103-Lab5

(kali㉿kali)-[~/CIP-B103-Lab5]
$ pwd
/home/kali/CIP-B103-Lab5

(kali㉿kali)-[~/CIP-B103-Lab5]
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported
```

Screenshot 1: Evidence folder structure created (mkdir, cd, pwd, find).

```
(kali㉿kali)-[~/CIP-B103-Lab5]
$ sudo apt install -y wireshark tshark python3-scapy net-tools
[sudo] password for kali:
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
net-tools is already the newest version (2.10-2).
The following packages were automatically installed and are no longer required:
  libraw23t64 libvidstab1.1 openjdk-21-jre openjdk-21-jre-headless
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1742

(kali㉿kali)-[~/CIP-B103-Lab5]
$ wget -O evidence/arp.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
--2026-08-01 10:26:01-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com) ... 185.199.109.133, 185.199.110.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443 ... connect
```

Screenshot 2: wireshark, tshark, python3-scapy, net-tools installed and arp.pcap downloaded.

```
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap      100%[=====>]  1.05K  --.-KB/s   in 0s
2026-08-01 10:26:01 (15.3 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

(kali㉿kali)-[~/CIP-B103-Lab5]
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(kali㉿kali)-[~/CIP-B103-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap \
| tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo          UNKNOWN    127.0.0.1/8  ::1/128
eth0        UP         192.168.44.128/24  fe80::fcd5:733e:9de1:56cb/64
docker0     DOWN      172.17.0.1/16
```

Screenshot 3: arp.pcap saved, hashed, and initial interface addresses listed.

```
(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.44.2 dev eth0 proto dhcp src 192.168.44.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.44.0/24 dev eth0 proto kernel scope link src 192.168.44.128 metric 100

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.44.254 dev eth0 lladdr 00:50:56:e1:c4:05 STALE
192.168.44.2 dev eth0 lladdr 00:50:56:f4:fd:61 REACHABLE
```

Screenshot 4: Route table and initial ARP (neighbor) table.

```
(kali㉿kali)-[~/CIP-B103-Lab5]
$ tree
.
├── evidence
│   └── arp.pcap
├── exported
├── reports
│   ├── arp_capture_hashes.txt
│   ├── arp_table_initial.txt
│   ├── interfaces.txt
│   └── routes.txt
├── screenshots
├── scripts
├── working
│   └── arp_working.pcap
└── 7 directories, 6 files

(kali㉿kali)-[~/CIP-B103-Lab5]
$
```

Screenshot 5: Evidence folder tree after Part preparation.

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	CIP-B103-Lab5-Fuseini Imoru Kantuogaa
Trainee name	Fuseini Imoru Kantuogaa
Date and time started	1 August 2026, approx. 10:26 local (Kali VM) — timestamp of the wget download of the evidence file
Evidence file name(s)	arp.pcap (working copy: arp_working.pcap); locally generated evidence/normal_arp.pcapng (Part A); evidence/controlled_arp_poison.pcapng attempted but incomplete (Part D, see notes)

Field	Student Entry
Source or generation method	arp.pcap downloaded via wget from the frankwxu digital-forensics-lab GitHub repository (a historical supplied ARP-spoofing sample, internally timestamped 11 Mar 2023); normal_arp.pcapng captured locally with tshark on eth0 while pinging the default gateway
Original SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e (evidence/arp.pcap)
Working-copy SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e (working/arp_working.pcap) — identical to original
Analysis workstation/VM	Kali Linux (user: kali), CIP-B103-Lab5 working directory, host-only interface eth0 (192.168.44.128/24, gateway 192.168.44.2)
Notes on any changes	cp --preserve=timestamps used for the working copy; SHA-256 matched exactly. The optional Part D controlled simulation hit a permission error writing directly to evidence/ and was redirected to /tmp; the screenshots do not show scripts/arp.py actually being run or a resulting changed ARP mapping before the trainee moved on to the restoration commands — Part D should be treated as attempted but inconclusive.

8. Part A - Observe Normal ARP Resolution

1. Identify the victim IP, gateway IP and your interface name.
2. Clear only the specific training neighbor entry where permitted.
3. Start an ARP-only capture, ping the gateway once and stop the capture.
4. Identify the broadcast request and unicast reply.

```
IFACE=eth0
```

```
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
```

```
sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
```

```
sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &
```

```
sleep 2
```

```
ping -c 1 "$GATEWAY_IP"
```

```
wait
```

```
ip neigh show | tee reports/arp_table_after_ping.txt
```



```

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip -br address
lo                UNKNOWN    127.0.0.1/8 ::1/128
eth0              UP          192.168.44.128/24 fe80::fcd5:733e:9de1:56cb/64
docker0           DOWN       172.17.0.1/16

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip route
default via 192.168.44.2 dev eth0 proto dhcp src 192.168.44.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.44.0/24 dev eth0 proto kernel scope link src 192.168.44.128 metric 100

(kali㉿kali)-[~/CIP-B103-Lab5]
$ IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')

(kali㉿kali)-[~/CIP-B103-Lab5]
$ echo $IFACE
eth0
$ echo $GATEWAY_IP
192.168.44.2

```

Screenshot 6: Interface/route recheck, IFACE and GATEWAY_IP variables set.

```

(kali㉿kali)-[~/CIP-B103-Lab5]
$ Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'

[4] done sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w /tmp/normal_arp.pcapng
(kali㉿kali)-[~/CIP-B103-Lab5]
$ ping -c 1 "$GATEWAY_IP"
PING 192.168.44.2 (192.168.44.2) 56(84) bytes of data:
64 bytes from 192.168.44.2: icmp_seq=1 ttl=128 time=0.424 ms

— 192.168.44.2 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.424/0.424/0.424/0.000 ms

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ls -lh /tmp/normal_arp.pcapng
-rw-r--r-- 1 root root 336 Aug 1 10:38 /tmp/normal_arp.pcapng

(kali㉿kali)-[~/CIP-B103-Lab5]
$ sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"

```

Screenshot 7: Normal ARP capture started, gateway pinged, and ARP entry flushed beforehand.

```

(kali㉿kali)-[~/CIP-B103-Lab5]
$ sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w /tmp/normal_arp.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
2

(kali㉿kali)-[~/CIP-B103-Lab5]
$ sudo tshark -i "$IFACE" -f 'arp' -a duration:20 -w /tmp/normal_arp.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
12

(kali㉿kali)-[~/CIP-B103-Lab5]
$ sudo cp /tmp/normal_arp.pcapng evidence/
sudo chown kali:kali evidence/normal_arp.pcapng

(kali㉿kali)-[~/CIP-B103-Lab5]
$ tshark -r evidence/normal_arp.pcapng -Y arp
1 0.0000000000 VMware_87:06:29 → VMware_f4:fd:61 ARP 42

```

Screenshot 8: Capture copied to evidence/ and tshark -Y arp showing the broadcast request and reply (plus background traffic).

```

Who has 192.168.44.2? Tell 192.168.44.128
2 0.000867778 VMware_f4:fd:61 → VMware_87:06:29 ARP 60
192.168.44.2 is at 00:50:56:f4:fd:61
3 6.796966003 VMware_c0:00:08 → Broadcast ARP 60 Who
has 192.168.44.2? Tell 192.168.44.1
4 7.706171743 VMware_c0:00:08 → Broadcast ARP 60 Who
has 192.168.44.2? Tell 192.168.44.1
5 8.583826839 VMware_c0:00:08 → Broadcast ARP 60 Who
has 192.168.44.2? Tell 192.168.44.1
6 9.486958782 VMware_c0:00:08 → Broadcast ARP 60 Who
has 192.168.44.2? Tell 192.168.44.1
7 10.419525199 VMware_c0:00:08 → Broadcast ARP 60 Wh
o has 192.168.44.2? Tell 192.168.44.1
8 11.289932214 VMware_c0:00:08 → Broadcast ARP 60 Wh
o has 192.168.44.2? Tell 192.168.44.1
9 12.195445674 VMware_c0:00:08 → Broadcast ARP 60 Wh
o has 192.168.44.2? Tell 192.168.44.1
10 13.128577471 VMware_c0:00:08 → Broadcast ARP 60 Wh
o has 192.168.44.2? Tell 192.168.44.1
11 13.991450478 VMware_c0:00:08 → Broadcast ARP 60 Wh
o has 192.168.44.2? Tell 192.168.44.1

```

Screenshot 9: Continued arp listing — repeated broadcast requests from a second host for the gateway IP.

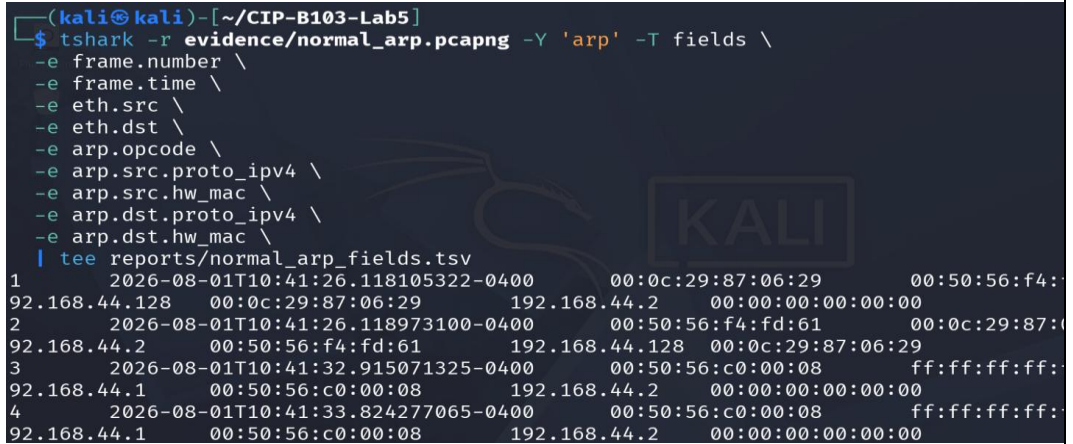
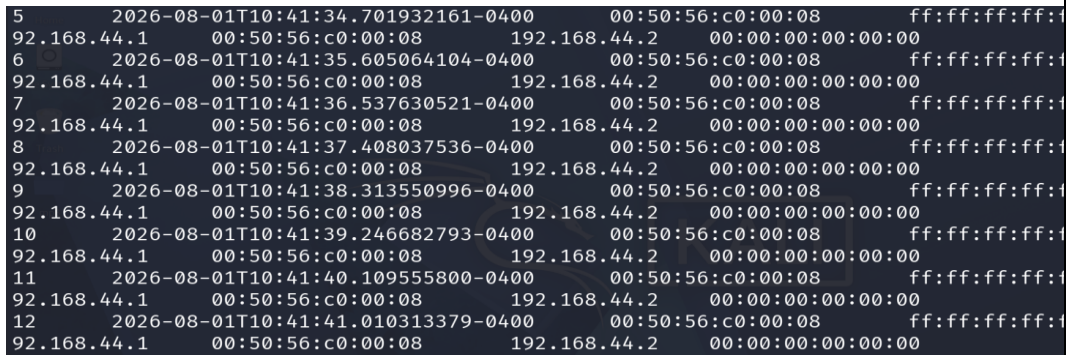
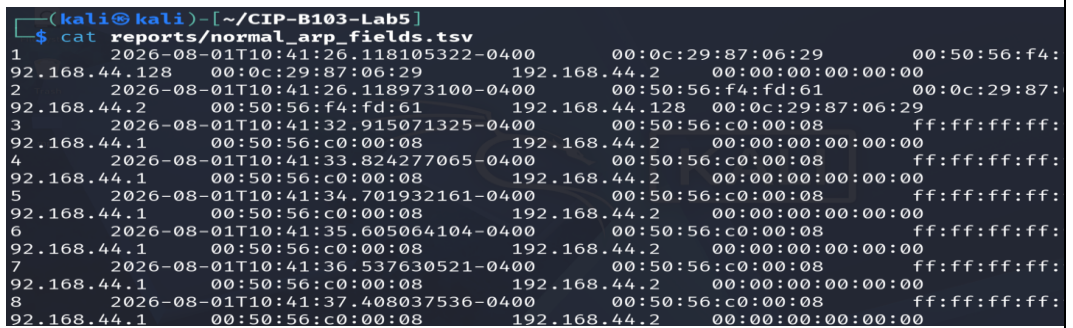
9. Part B - Analyze ARP Request and Reply Fields

```

tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/normal_arp_fields.tsv

```

Field	Normal Request	Normal Reply
Ethernet destination	ff:ff:ff:ff:ff:ff	Requester MAC
ARP opcode	1	2
Sender protocol address	Requester IP	Target IP
Sender hardware address	Requester MAC	Target MAC
Target protocol address	Target IP	Requester IP

Field	Normal Request	Normal Reply
Forensic interpretation	Who has target IP?	Target IP is at target MAC.  <i>Screenshot 10: Field extraction of the normal ARP capture (request and reply, start).</i>  <i>Screenshot 11: Field extraction continued — repeated background broadcast requests.</i>  <i>Screenshot 12: reports/normal_arp_fields.tsv contents confirmed via cat.</i>

10. Part C - Analyze the Supplied Poisoning Capture

```
PCAP=working/arp_working.pcap
```

```
# Inventory all ARP claims
```

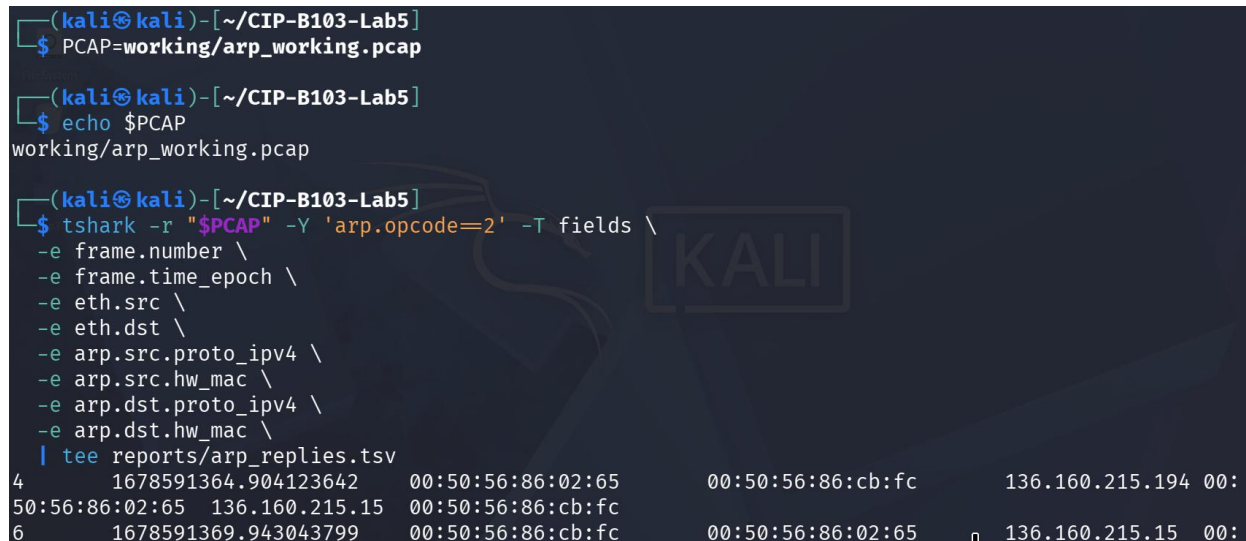
```
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv
```

```
# Summarize IP-to-MAC claims
```

```
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
```

```
# Look for gratuitous or unsolicited-looking replies
```

```
tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
  -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
  | tee reports/unicast_arp_replies.tsv
```



```
(kali㉿kali)-[~/CIP-B103-Lab5]
$ PCAP=working/arp_working.pcap

(kali㉿kali)-[~/CIP-B103-Lab5]
$ echo $PCAP
working/arp_working.pcap

(kali㉿kali)-[~/CIP-B103-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number \
  -e frame.time_epoch \
  -e eth.src \
  -e eth.dst \
  -e arp.src.proto_ipv4 \
  -e arp.src.hw_mac \
  -e arp.dst.proto_ipv4 \
  -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:
50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15 00:
```

Screenshot 13: Supplied arp.pcap analyzed — ARP reply inventory (arp.opcode==2).

```

6          1678591369.943043799    00:50:56:86:cb:fc    00:50:56:86:02:65    136.160.215.15    00:
50:56:86:cb:fc    136.160.215.194    00:50:56:86:02:65

(kali㉿kali)-[~/CIP-B103-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
| sort | uniq -c | sort -nr \
| tee reports/ip_mac_claims.txt
    1 136.160.215.194 00:50:56:86:02:65
    1 136.160.215.15  00:50:56:86:cb:fc

(kali㉿kali)-[~/CIP-B103-Lab5]
$ tshark -r "$PCAP" \
-Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' \
-T fields \
-e frame.number \
-e frame.time \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e eth.dst \
| tee reports/unicast_arp_replies.tsv

```

Screenshot 14: IP-to-MAC claim summary and start of the unicast-reply filter.

```

4          2023-03-11T22:22:44.904123642-0500    136.160.215.194    00:50:56:86:02:65    00:50:56:86
:cb:fc
6          2023-03-11T22:22:49.943043799-0500    136.160.215.15    00:50:56:86:cb:fc    00:50:56:86
:02:65

(kali㉿kali)-[~/CIP-B103-Lab5]
$ cat reports/ip_mac_claims.txt
    1 136.160.215.194 00:50:56:86:02:65
    1 136.160.215.15  00:50:56:86:cb:fc

(kali㉿kali)-[~/CIP-B103-Lab5]
$

```

Screenshot 15: Unicast ARP replies confirmed and ip_mac_claims.txt contents.

11. Part D - Optional Controlled Host-Only Simulation

Instructor Approval Required

Skip this section unless the virtual network is confirmed as host-only/internal and the victim, gateway and analyst IP addresses are assigned specifically for the lab.

5. Record clean ARP tables on victim and gateway.
6. Start ARP capture on the analyst VM.
7. Run the instructor-provided arp.py for no more than 30 seconds.
8. Capture the changed victim mapping and stop the script.
9. Restore correct ARP mappings or restart the isolated VMs.

Example placeholders only - replace with instructor-assigned lab IPs

VICTIM_IP=192.168.56.20

GATEWAY_IP=192.168.56.1

IFACE=eth0

sudo tshark -i "\$IFACE" -f 'arp' -a duration:40 -w evidence/controlled_arp_poison.pcapng &
sleep 2

In another terminal, run the instructor-provided script for <=30 seconds:

sudo timeout 30 python3 scripts/arp.py "\$VICTIM_IP" "\$GATEWAY_IP"

wait

ip neigh show | tee reports/analyst_arp_table_after.txt

```
(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip -br address
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0              UP          192.168.44.128/24 fe80::fcd8:733
docker0          DOWN        172.17.0.1/16

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip route
default via 192.168.44.2 dev eth0 proto dhcp src 192.168.44.128
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
192.168.44.0/24 dev eth0 proto kernel scope link src 192.168.44.

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip neigh show | tee reports/arp_table_before.txt
192.168.44.254 dev eth0 lladdr 00:50:56:e1:c4:05 STALE
192.168.44.2 dev eth0 lladdr 00:50:56:f4:fd:61 REACHABLE

(kali㉿kali)-[~/CIP-B103-Lab5]
$ IFACE=eth0

(kali㉿kali)-[~/CIP-B103-Lab5]
$ sudo tshark -i "$IFACE" -f 'arp' \
-a duration:40 \
-w evidence/controlled_arp_poison.pcapng &
[4] 32346

(kali㉿kali)-[~/CIP-B103-Lab5]
$ Running as user "root" and group "root". This could be dange
Capturing on 'eth0'
tshark: The file to which the capture would be saved ("evidence/
```

Screenshot 16: Pre-simulation ARP table recorded; controlled capture attempted (initial permission error writing to evidence/).

```

(kali㉿kali)-[~/CIP-B103-Lab5]
$ Running as user "root" and group "root". This could be dange
(kali㉿kali)-[~/CIP-B103-Lab5]
6 $

[4] done      sudo tshark -i "$IFACE" -f 'arp' -a duration:4
0 -w
(kali㉿kali)-[~/CIP-B103-Lab5]
$ sudo tshark -i "$IFACE" -f 'arp' \
-a duration:40 \
-w /tmp/controlled_arp_poison.pcapng &
[4] 33281

(kali㉿kali)-[~/CIP-B103-Lab5]
$ Running as user "root" and group "root". This could be dange
rous.
Capturing on 'eth0'

[4] done      sudo tshark -i "$IFACE" -f 'arp' -a duration:4
0 -w
(kali㉿kali)-[~/CIP-B103-Lab5]
$

```

Screenshot 17: Controlled capture retried, writing to /tmp/controlled_arp_poison.pcapng.

12. Part E - Detection Logic and Timeline

Time	Claimed IP	Claimed MAC	Target	Request Seen First?	Assessment
2023-03-11 22:22:44.904-0500 (frame 4)	136.160.215.194	00:50:56:86:02:65	136.160.215.15 (unicast reply, eth.dst = 00:50:56:86:cb:fc)	No corresponding request visible in this short excerpt	Unsolicited/gratuitous-style unicast reply; only one MAC claimed this IP in the sample — no second conflicting claim was captured, so this alone is suggestive but not conclusive proof of poisoning.

Time	Claimed IP	Claimed MAC	Target	Request Seen First?	Assessment
2023-03-11 22:22:49.943 -0500 (frame 6), ~5.04 s later	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194 (unicast reply, eth.dst = 00:50:56:86:02:65)	No corresponding request visible in this short excerpt	Reciprocal unsolicited unicast reply from the other host; the two replies together form a classic bidirectional ARP-spoofing pattern (each host told the other party's MAC directly), though again no dual-MAC conflict is present in this 8-frame sample.
Summary across evidence/arp.pcap	Only 2 distinct IP-to-MAC claims total (1 each)	n/a	n/a	n/a	reports/ip_mac_claims.txt shows exactly one claim per IP — this short supplied capture illustrates the unicast-reply technique used in ARP spoofing but does not itself contain a repeated/conflicting claim; a fuller capture would be needed to confirm an active poisoning campaign.

Time	Claimed IP	Claimed MAC	Target	Request Seen First?	Assessment
Part D (controlled simulation)	Not captured	Not captured	Not captured	n/a	The optional host-only simulation was started but scripts/arp.py does not appear to have been executed in the supplied screenshots, and reports/analyst_arp_table_after.txt was not produced — no live poisoning evidence is available from Part D in this submission.

- Flag a gateway IP that changes from its legitimate MAC to the analyst/attacker MAC.
- Flag repeated ARP replies without a corresponding recent request.
- Flag one MAC claiming both victim and gateway IP addresses.
- Correlate ARP evidence with packet forwarding, DNS/HTTP anomalies and endpoint ARP-cache changes.

13. Restoration and Prevention

Stop any training script first, then clear learned entries on the lab VM

```
sudo ip neigh flush all
```

Renew/reconnect the isolated interface as appropriate

```
ip neigh show
```

Confirm no ARP-poisoning process remains

```
ps aux | grep -E '[a]rp.py|[s]capy' | tee reports/process_check.txt
```

- Use Dynamic ARP Inspection with DHCP snooping on managed switches where available.
- Segment networks, enable port security and monitor gateway IP-to-MAC consistency.
- Use encrypted application protocols so interception does not expose plaintext.
- Deploy ARP monitoring tools and correlate alerts with switch and endpoint logs.

```

(kali㉿kali)-[~/CIP-B103-Lab5]
$ sudo ip neigh flush all

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip neigh show
192.168.44.2 dev eth0 lladdr 00:50:56:f4:fd:61 REACHABLE

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ping -c 1 192.168.44.2
PING 192.168.44.2 (192.168.44.2) 56(84) bytes of data:
64 bytes from 192.168.44.2: icmp_seq=1 ttl=128 time=0.264 ms

— 192.168.44.2 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.264/0.264/0.264/0.000 ms

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ip neigh show
192.168.44.2 dev eth0 lladdr 00:50:56:f4:fd:61 REACHABLE

(kali㉿kali)-[~/CIP-B103-Lab5]
$ ps aux | grep -E '[a]rp.py|[s]capy' | tee reports/process_check.txt

(kali㉿kali)-[~/CIP-B103-Lab5]

```

Screenshot 18: Restoration — ip neigh flush all, ARP table re-verified, gateway re-pinged, and no arp.py/scapy process remaining.

Required Screenshots Checklist

- ☐ Initial interface, route and ARP table.
- ☐ Normal broadcast ARP request.
- ☐ Normal ARP reply.
- ☐ Supplied capture hash.
- ☐ ARP reply inventory.
- ☐ IP-to-MAC claim summary.
- ☐ Conflicting gateway MAC evidence.
- ☐ Clean versus poisoned ARP table comparison.
- ☐ Controlled capture, if authorized.
- ☐ Restored ARP table and process check.

Student Report Template

10. Executive summary and authorization.
11. Network diagram and assigned IP/MAC addresses.
12. Evidence integrity.
13. Normal ARP behaviour.
14. Poisoning indicators and timeline.

15. ARP-table comparison.
16. Impact assessment.
17. Restoration and preventive controls.
18. Appendix.

Submission Requirements

- One PDF report named CIP-B103-Lab5_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Safety and environment	15	Host-only scope and restoration documented.
Normal ARP analysis	15	Request/reply fields and ARP-cache behaviour.
Poisoning detection	25	Conflicting claims, unsolicited replies and timeline.
Controlled evidence or supplied PCAP analysis	20	Complete, repeatable capture analysis.
Mitigation and restoration	15	Correct cleanup and preventive controls.
Report quality	10	Clear diagram, screenshots and conclusions.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
ARP request opcode	1.
ARP reply opcode	2.
Broadcast destination	ff:ff:ff:ff:ff:ff for a typical request.
Poisoning indicator	An IP, especially the gateway, is claimed by a different MAC.
Unsolicited reply	ARP reply appears without a corresponding request.
MITM pattern	Attacker MAC is inserted for victim and/or gateway IP mappings.

Command Reference Summary

Command	Purpose
ip neigh show	Display Linux neighbor/ARP cache.
arp -a	Display legacy ARP table.
arp.opcode==1	Filter ARP requests.
arp.opcode==2	Filter ARP replies.
arp.src.proto_ipv4	Claimed sender IP.
arp.src.hw_mac	Claimed sender MAC.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.